Count Islands With Total Value Divisible by K

Problem Statement

3619. Count Islands With Total Value Divisible by K

You are given an m x n matrix grid and a positive integer k. An island is a group of positive integers (representing land) that are 4-directionally connected horizontally or vertically.

The total value of an island is the sum of the values of all cells in the island.

Return the number of islands with a total value divisible by k.

Example 1:
Input: grid = [[0,2,1,0,0],[0,5,0,0,5],[0,0,1,0,0],[0,1,4,7,0],[0,2,0,0,8]], k = 5
Output: 2
Explanation: The grid contains four islands. Two islands have a total value divisible by 5.

Example 2:
Input: grid = [[3,0,3,0],[0,3,0,3],[3,0,3,0]], k = 3
Output: 6
Explanation: The grid contains six islands, each with a total value divisible by 3.

Constraints:
m == grid.length
n == grid[i].length
1 <= m, n <= 1000
1 <= m * n <= 10^5
0 <= grid[i][j] <= 10^6
1 <= k <= 10^6

Approach

Use DFS to find every island of positive cells. While visiting an island, calculate the sum of all its cell values and mark each visited cell as 0 so it is not counted again.

For every new positive cell found during the scan:
1. Start a DFS from that cell.
2. Add the cell value to the island sum.
3. Mark the cell as 0.
4. Recursively visit its four-directionally connected positive neighbors.
5. After the DFS completes, check whether the island sum is divisible by k.
6. If it is divisible, increase the answer by 1.

Use long for the island sum because an island can contain many cells, each with a value up to 10^6.

Example:

For grid = [[0,2,1,0,0],[0,5,0,0,5],[0,0,1,0,0],[0,1,4,7,0],[0,2,0,0,8]] and k = 5:
- One island has sum 8, which is divisible by 5.
- Another island has sum 20, which is divisible by 5.
- The remaining two islands are not divisible by 5.
Therefore, the answer is 2.

Java Solution

```
class Solution {
    private final int[] rowDirections = {-1, 1, 0, 0};
    private final int[] columnDirections = {0, 0, -1, 1};

    public int countIslands(int[][] grid, int k) {
        int rows = grid.length;
        int columns = grid[0].length;

        int divisibleIslands = 0;

        for (int row = 0; row < rows; row++) {
            for (int column = 0; column < columns; column++) {
                if (grid[row][column] > 0) {
                    long islandSum = calculateIslandSum(grid, row, column);

                    if (islandSum % k == 0) {
                        divisibleIslands++;
                    }
                }
            }
        }

        return divisibleIslands;
    }

    private long calculateIslandSum(int[][] grid, int row, int column) {
        if (row < 0 || row >= grid.length
                || column < 0 || column >= grid[0].length) {
            return 0;
        }

        if (grid[row][column] == 0) {
            return 0;
        }

        long islandSum = grid[row][column];
        grid[row][column] = 0;

        for (int direction = 0; direction < 4; direction++) {
            islandSum += calculateIslandSum(
                    grid,
                    row + rowDirections[direction],
                    column + columnDirections[direction]);
        }

        return islandSum;
    }
}
```

Complexity

Time Complexity: O(m * n), since every cell is visited at most once.

Space Complexity: O(m * n) in the worst case due to the DFS recursion stack.